

OnDeviceTraining

Code Reading Guide

A file-by-file walkthrough of the full C codebase —
reading order, purpose of each file, and how they connect

Mohamed · Master's Thesis · 2026

How to Use This Guide

The codebase is structured in layers, from lowest-level primitives up to the full training loop. Each file listed below builds on the ones before it. Reading in order means you will always understand every type and function you encounter before it is used. Each entry shows the header (.h) and implementation (.c) as a pair — read the header first to understand the contract, then the .c to see how it is fulfilled.

Phase	Module	What it provides
1 — Foundation	Common, DTypes, Rounding	Debug macros, byte I/O, numeric rounding modes
2 — Memory	DataStorage, StorageApi	Static arena allocator — no heap malloc anywhere
3 — Tensor	Tensor, Quantization, TensorConversion, TensorApi, QuantizationApi	Core data structure, quantization configs, dtype conversion, user-facing init helpers
4 — Arithmetic	Arithmetic, Matmul, Add/Sub/Mul/Div/Log/Sum/Square/MinMax/Comparison/Distributions	All element-wise and matrix math ops, weight initialization distributions
5 — RNG	RNG	Pseudo-random number generator for shuffle and stochastic rounding
6 — Layers	Layer, Linear, Relu, Softmax, QuantizationLayer, Conv1d, LinearApi/ReluApi/SoftmaxApi	Forward + backward pass for every layer type; user-facing layer constructors
7 — Loss	LossFunction, CrossEntropy, MSE	Loss function dispatch table and both implemented losses
8 — Optimizer	Optimizer, Sgd, OptimizerApi, SgdApi	SGD with optional momentum; optimizer state management

9 — Data I/O	Dataset, DataLoader, NPYLoader, DataLoaderApi, NPYLoaderApi, CSVHelper	Structs for samples/batches, .npy file loading, CSV logging
10 — Serialization	Serialize, Deserialize	Binary save/load of tensors, parameters, and whole models
11 — Inference	InferenceApi	Forward-only pass, batched inference, inference-with-loss
12 — Training Loop	TrainingLoopApi, CalculateGradsSequential, TrainingBatchDefault, TrainingEpochDefault	Full training run orchestration: epoch loop, batch loop, backprop, eval
13 — Experiment	MnistExperiment.c	End-to-end MNIST validation tying all layers together

Phase 1

Foundation

*Debug macros · byte serialisation
· rounding*

1

Common.h (header only)

`src/common/include/Common.h`

The single most-included file in the whole project. Defines the three-level debug print macros: `PRINT_DEBUG` (level 3), `PRINT_INFO` (level 2), `PRINT_ERROR` (level 1), and `PRINT_BYTE_ARRAY` for raw byte dumps. The active level is set at compile time via `-DDEBUG_MODE_DEBUG / INFO / ERROR`. Every other `.c` file includes this first.

Reads from: Nothing — no dependencies.

Key types/functions: `PRINT_DEBUG`, `PRINT_INFO`, `PRINT_ERROR`, `PRINT_BYTE_ARRAY`

Note: Read this before anything else. You will see these macros in every `.c` file you open.

2

Rounding.h / Rounding.c

`src/arithmetic/include/Rounding.h` + `src/arithmetic/Rounding.c`

Defines the `roundingMode_t` enum (`HTE` = Half-to-Even, `SRHTE` = Stochastic Rounding Half-to-Even) and two functions: `roundByMode(float, mode) -> int32` which converts a float to an integer using the chosen rounding strategy, and `clamp(float, min, max)` which saturates a value to a range. Both are essential for the FQT quantization path.

Reads from: Nothing.

Key types/functions: `roundingMode_t`, `roundByMode()`, `clamp()`

Note: Stochastic rounding (`SRHTE`) is a key ingredient of Fully Quantized Training — it prevents gradient underflow during `int8` backward passes.

3

DTypes.h / DTypes.c[src/tensor/include/DTypes.h](#) + [src/tensor/DTypes.c](#)

Low-level byte I/O utilities. Provides functions to read and write int32 and float32 values from/to raw uint8_t byte arrays, in both single-value and array forms: `readBytesAsInt32()`, `readBytesAsFloat()`, `writeInt32ToByteArray()`, `writeFloatToByteArray()`, and their array variants. Used by the serialization module to pack model weights into a portable binary format.

Reads from: Nothing (pure byte manipulation).

Key types/functions: `readBytesAsInt32()`, `readBytesAsFloat()`, `writeInt32ToByteArray()`, `writeFloatToByteArray()`

Phase 2

Memory Management

Static arena allocator — zero heap malloc

4

DataStorage.h / DataStorage.c[src/tensor/include/DataStorage.h](#) + [src/tensor/DataStorage.c](#)

Implements a `dataStorage_t` struct: a flat uint8_t buffer plus an array of `dataEntry_t` records that each hold a pointer and an element count. Provides `getDataFromStorage()` and `addDataToStorage()`. This is the low-level backing store for the arena.

Reads from: `DTypes.h` (indirect).

Key types/functions: `dataStorage_t`, `dataEntry_t`, `getDataFromStorage()`, `addDataToStorage()`

5

StorageApi.h / StorageApi.c[src/userApi/include/StorageApi.h](#) + [src/userApi/StorageApi.c](#)

The single most important architectural decision in the codebase: `reserveMemory(size)` and `freeReservedMemory(ptr)`. All dynamic allocations in the project go through these two calls. On a microcontroller they map to a static arena; on a host PC they wrap `malloc/free`. This means there is zero heap fragmentation risk on the MCU.

Reads from: `DataStorage.h`.

Key types/functions: `reserveMemory()`, `freeReservedMemory()`

Note: Every subsequent module calls `reserveMemory()` instead of `malloc()`. Read this file before looking at any initializer function.

Phase 3

Tensor System

*Core data structure · quantization
· type conversion · user API*

6

Quantization.h / Quantization.c

`src/tensor/include/Quantization.h + src/tensor/Quantization.c`

Defines the `quantization_t` struct and its five flavours: INT32 (raw 32-bit integer), FLOAT32 (native float, no quantization), SYM_INT32 (symmetric quantization with int32 accumulators), SYM (symmetric int8), and ASYM (asymmetric int8 with zero-point). Each flavour has its own config struct (scale, qBits, zeroPoint, roundingMode). Provides init helpers: `initFloat32Quantization()`, `initSymQuantization()`, etc.

Reads from: *Rounding.h*.

Key types/functions: `quantization_t`, `qtype_t`, `symQConfig_t`, `asymQConfig_t`, `initFloat32Quantization()`

7

Tensor.h / Tensor.c

`src/tensor/include/Tensor.h + src/tensor/Tensor.c`

Defines `tensor_t` — the central data structure. A tensor holds: a `uint8_t*` data pointer, a `shape_t*` (dimensions + axis ordering), a `quantization_t*`, and an optional `sparsity_t*` (stub, not yet populated). Also defines `parameter_t` which pairs a weight tensor with its gradient tensor. Key utility functions include: `calcNumberOfElementsByTensor()`, `calcBytesPerTensor()`, `transposeTensor()`, `setTensorValues()`, `setParameterValues()`, `copyTensor()`, `printTensor()`.

Reads from: *Quantization.h*, *StorageApi.h*.

Key types/functions: `tensor_t`, `parameter_t`, `shape_t`, `sparsity_t`, `transposeTensor()`, `calcBytesPerTensor()`

Note: The `sparsity_t` struct is defined here and is the hook point for the upcoming RigL sparse mask implementation.

8

TensorConversion.h / TensorConversion.c

`src/tensor/include/TensorConversion.h + src/tensor/TensorConversion.c`

Implements a 5x5 conversion dispatch table (`conversionMatrix[from_qtype][to_qtype]`) mapping every quantization-type pair to a conversion function. The public API is `convertTensor(input, output)` which looks up the right converter and applies it. This handles float->int8, int8->float, int32->int8, etc., all in one place.

Reads from: *Tensor.h*, *Quantization.h*.

Key types/functions: `convertTensor()`, `conversionMatrix[5][5]`, `conversionFunction_t`

9

QuantizationApi.h / QuantizationApi.c

`src/userApi/tensor/include/QuantizationApi.h + src/userApi/tensor/QuantizationApi.c`

User-facing quantization constructors that call `reserveMemory()` and return heap-allocated `quantization_t` pointers. Four functions: `quantizationInitFloat()`, `quantizationInitInt32()`, `quantizationInitSymInt32(roundingMode)`, `quantizationInitAsym(qBits, roundingMode)`. These are the entry points you use when building a model.

Reads from: *QuantizationApi.h, StorageApi.h, Quantization.h.*

Key types/functions: *quantizationInitFloat(), quantizationInitSymInt32(), quantizationInitAsym()*

10

TensorApi.h / TensorApi.c

`src/userApi/tensor/include/TensorApi.h + src/userApi/tensor/TensorApi.c`

The richest API file in the project. Provides high-level tensor constructors: `tensorInitFloat()`, `tensorInitInt32()`, `tensorInitSymInt32()`, `tensorInitAsym()`, `tensorInit()`, and the important `tensorInitWithDistribution()` which initialises weights with Xavier/Kaiming/Zeros/Ones distributions. Also provides `gradInitFloat()`, `gradInitSymInt32()`, `parameterInit()`, `getTensorLike()`, `getQLike()`, and all the `free*()` functions (`freeTensor`, `freeParameter`, `freeShape`, `freeData`, `freeQuantization`).

Reads from: *TensorApi.h, Tensor.h, QuantizationApi.h, Distributions.h, StorageApi.h.*

Key types/functions: *tensorInitWithDistribution(), gradInitFloat(), parameterInit(), getTensorLike(), freeTensor()*

Phase 4

Arithmetic

All math operations on tensors

11

Arithmetic.h / Arithmetic.c

`src/arithmetic/include/Arithmetic.h + src/arithmetic/Arithmetic.c`

The arithmetic dispatch engine. Defines two function-pointer typedefs — `int32ElementArithmeticFunc_t` and `floatElementArithmeticFunc_t` — and provides generic point-wise arithmetic drivers: `int32PointWiseArithmetic()`, `floatPointWiseArithmetic()`, and their in-place variants. Also provides scalar-broadcast variants (`int32ElementWithTensorArithmetic`). All Add, Sub, Mul, Div, etc. are built on top of these. Also contains index helpers: `calcTensorIndexByIndices()`, `calcIndicesByRowIndex()`, `orderDims()`, `doDimensionsMatch()`.

Reads from: *Tensor.h.*

Key types/functions: *int32PointWiseArithmetic(), floatPointWiseArithmetic(), calcTensorIndexByIndices()*

File	What it provides
Add.h / Add.c	<code>addInt32Tensors()</code> , <code>addFloatTensors()</code> , <code>addInt32TensorsInplace()</code> , <code>addFloatTensorsInplace()</code> — element-wise addition
Sub.h / Sub.c	<code>subInt32Tensors()</code> , <code>subFloatTensors()</code> and in-place variants — element-wise subtraction

Mul.h / Mul.c	mulInt32Tensors(), mulFloatTensors() — element-wise multiplication (Hadamard product)
Div.h / Div.c	divInt32Tensors(), divFloatTensors() — element-wise division
Log.h / Log.c	logFloatTensor() — natural log of each element; used in cross-entropy loss
Sum.h / Sum.c	sumFloat(), sumInt32() — total reduction over all elements of a tensor
Square.h / Square.c	squareFloatTensor(), squareInt32Tensor() — element-wise squaring; used in MSE loss
MinMax.h / MinMax.c	minFloat(), maxFloat(), minInt32(), maxInt32() — global min/max reduction
Comparison.h / Comparison.c	argmax() — returns index of max element; used to compute accuracy during eval
Matmul.h / Matmul.c	matmulFloat32Tensors(), matmulInt32Tensors(), matmulSymInt32Tensors() — 2D matrix multiply; core of the linear layer forward and backward passes
Distributions.h / Distributions.c	randomNormal(), randomUniform(), xavierUniform(), xavierNormal(), kaimingUniform(), kaimingNormal() — weight initialisation distributions

Phase 5

Random Number Generator

Shuffle and stochastic rounding support

12

RNG.h / RNG.c

`src/rng/include/RNG.h + src/rng/RNG.c`

A lightweight, seedable pseudo-random number generator backed by a 32-bit state. Provides `rngSetSeed(uint32_t)`, `rngGetSeed()`, `rngNextFloat()` (returns a float in $[0,1]$), and `rngShuffleIndices(indices, n)` which Fisher-Yates shuffles an index array in-place. The DataLoader uses `rngShuffleIndices` for dataset shuffling; the SRHTE rounding mode uses `rngNextFloat` for stochastic rounding decisions.

Reads from: Nothing.

Key types/functions: `rng32_t`, `rngSetSeed()`, `rngNextFloat()`, `rngShuffleIndices()`

Phase 6

Layer System

Forward + backward for all layer types

13

Layer.h / Layer.c`src/layer/include/Layer.h + src/layer/Layer.c`

Defines the `layer_t` struct — the polymorphic base for all layers. A layer has a `layerType_t` (LINEAR, RELU, CONV1D, SOFTMAX, SEQUENTIAL, QUANTIZATION) and a `layerConfig_t` union that points to the type-specific config. Three function-pointer typedefs (`forwardFn_t`, `backwardFn_t`, `calcOutputShapeFn_t`) define the interface. A global `layerFunctions[]` dispatch table maps each `layerType_t` to its forward, backward, and shape-calc functions. `initLayer()` wires them together.

Reads from: *Tensor.h*.

Key types/functions: `layer_t`, `layerType_t`, `layerFunctions[]`, `forwardFn_t`, `backwardFn_t`, `initLayer()`

14

Linear.h / Linear.c`src/layer/include/Linear.h + src/layer/Linear.c`

The fully-connected (dense) layer. `linearConfig_t` holds `weight parameter_t`, `bias parameter_t`, and four `quantization_t` pointers (forward output, weight gradient, bias gradient, propagated loss). Provides float and `SymInt32` specialisations of both forward and backward: `linearForwardFloat()`, `linearForwardSymInt32()`, `linearBackward()`, and the individual gradient helpers `linearCalcWeightGradsFloat32()`, `linearCalcBiasGradsFloat32()`, `linearCalcPropLossFloat32()` (and `SymInt32` equivalents). Forward = $W \cdot x + b$; backward computes dL/dW , dL/db , and dL/dx .

Reads from: *Layer.h*, *Matmul.h*, *Add.h*, *Arithmetic.h*, *TensorConversion.h*.

Key types/functions: `linearConfig_t`, `linearForwardFloat()`, `linearForwardSymInt32()`, `linearCalcWeightGradsFloat32()`

15

Relu.h / Relu.c`src/layer/include/Relu.h + src/layer/Relu.c`

The ReLU activation layer. Stateless — no trainable parameters. `reluConfig_t` holds two quantization pointers. Forward: $\max(0, x)$ — in int8 this is a comparison against the zero-point, making it near-free. Backward: passes the upstream delta unchanged for positive inputs, zeroes it for negative inputs (mask from the forward pass).

Separate float and asymmetric int8 paths.

Reads from: *Layer.h*, *Tensor.h*, *Comparison.h*.

Key types/functions: `reluConfig_t`, `reluForwardFloat()`, `reluBackwardFloat()`, `reluForwardAsym()`

16

Softmax.h / Softmax.c`src/layer/include/Softmax.h + src/layer/Softmax.c`

Softmax output layer. `softmaxConfig_t` holds forward and backward quantization pointers. Forward: $\exp(x_i) / \sum(\exp(x_j))$ using the Log module for numerical stability. Backward: the Jacobian-vector product $dL/dx = \text{softmax} * (dL/dy - \text{dot}(dL/dy, \text{softmax}))$. In practice the backward is always fused with cross-entropy (`crossEntropySoftmaxBackward`) which simplifies to $y_{\text{hat}} - y_{\text{true}}$.

Reads from: *Layer.h, Log.h, Sum.h, Sub.h, Mul.h.*

Key types/functions: `softmaxConfig_t`, `softmaxForward()`, `softmaxBackward()`

17

QuantizationLayer.h / QuantizationLayer.c`src/layer/include/QuantizationLayer.h + src/layer/QuantizationLayer.c`

A pass-through quantization boundary layer. `quantizationForward()` converts a tensor to the target quantization (e.g. float32 -> int8) using `convertTensor()`. `quantizationBackward()` propagates gradients back through the quantization boundary (straight-through estimator). This layer is the insertion point for Fully Quantized Training: it sits between float inputs and int8 compute.

Reads from: *Tensor.h, TensorConversion.h.*

Key types/functions: `quantizationForward()`, `quantizationBackward()`

Note: This is the key FQT hook. Wiring the backward pass of this layer with proper rounding (SRHTE) is Milestone C8.

18

Conv1d.h / Conv1dTransposed.h + .c`src/layer/include/Conv1d.h + src/layer/Conv1d.c`

1-D convolution over temporal sequences — designed for MFCC feature maps. The header is currently a stub (the `ENV5_RUNTIME_CONV1D_H` guard with no declarations), indicating the interface is planned but not yet populated. `Conv1dTransposed.c` provides the transposed convolution variant. Read the header to see where the API will live; this is a future extension point.

Reads from: *Layer.h, Arithmetic.h.*

Key types/functions: *(stub — no public API yet)*

Note: The Conv1d forward/backward will follow the same pattern as Linear.

Layer User-API — Constructors

After reading the core layer files, read the three user-facing constructors in `src/userApi/layer/`. They call `reserveMemory()` to allocate the configs and wire them into `layer_t` structs ready for use:

File	Function	What it does
<code>LinearApi.h / .c</code>	<code>linearLayerInit(weights, bias, fwdQ, wGradQ, bGradQ, propQ)</code>	Allocates a <code>linearConfig_t</code> , sets all 4 quantizations, returns a <code>layer_t*</code> . <code>linearLayerInitNonTrainable()</code> omits gradient setup (inference-only).
<code>ReluApi.h / .c</code>	<code>reluLayerInit(forwardQ, backwardQ)</code>	Allocates a <code>reluConfig_t</code> and wraps it in a <code>layer_t*</code> .

SoftmaxApi.h / .c	softmaxLayerInit(forwardQ, backwardQ)	Allocates a softmaxConfig_t and wraps it in a layer_t*.
-------------------	---------------------------------------	---

Phase 7

Loss Functions

MSE and Cross-Entropy with dispatch table

19

LossFunction.h / LossFunction.c

src/loss_functions/include/LossFunction.h +
src/loss_functions/LossFunction.c

Defines the lossType_t enum (MSE, CROSS_ENTROPY) and two function-pointer typedefs: lossFwdFn_t (returns float scalar loss) and lossBwdFn_t (fills a gradient tensor). A global lossFunctions[] dispatch table maps each lossType_t to its forward and backward functions, so the training loop can call loss by enum without knowing which loss is active.

Reads from: Tensor.h.

Key types/functions: lossType_t, lossFunctions_t, lossFunctions[]

20

CrossEntropy.h / CrossEntropy.c

src/loss_functions/include/CrossEntropy.h +
src/loss_functions/CrossEntropy.c

Implements cross-entropy loss. crossEntropyForwardFloat() computes $-\sum(y_{\text{true}} * \log(y_{\text{hat}}))$ using the Log and Sum modules. crossEntropySoftmaxBackward() computes the combined softmax+cross-entropy gradient $y_{\text{hat}} - y_{\text{true}}$ — numerically stable and $O(n)$ instead of $O(n^2)$. This is the loss used in the MNIST experiment and will be used for speaker identification.

Reads from: Tensor.h, Log.h, Sum.h.

Key types/functions: crossEntropyForwardFloat(), crossEntropySoftmaxBackward()

21

MSE.h / MSE.c

src/loss_functions/include/MSE.h + src/loss_functions/MSE.c

Mean Squared Error loss. mseLossForward() computes $\text{mean}((\text{output} - \text{label})^2)$. mseLossBackwardFloat() and mseLossBackwardAsym() compute $2 * (\text{output} - \text{label}) / N$. A dispatcher mseLossBackward() selects the right variant by quantization type. MSE is available for regression tasks or autoencoder experiments.

Reads from: Tensor.h, Sub.h, Square.h, Sum.h.

Key types/functions: mseLossForward(), mseLossBackwardFloat()

Phase 8

Optimizer

SGD with momentum · optimizer dispatch table

22

Optimizer.h / Optimizer.c`src/optimizer/include/Optimizer.h + src/optimizer/Optimizer.c`

Defines `optimizer_t` — the polymorphic optimizer. Holds a type enum (SGD or SGD_M), a `qtype_t`, an `optimImpl_t` union pointing to the concrete implementation, a `parameter_t**` array (all trainable parameters), a `states_t**` array (velocity buffers for momentum), and the total number of states. A global `optimizerFunctions[]` dispatch table maps each type to `step()` and `zero()` functions. `calcTotalNumberOfStates()` counts parameters across the model.

Reads from: `Tensor.h`, `Quantization.h`, `Layer.h`.

Key types/functions: `optimizer_t`, `states_t`, `optimizerFunctions[]`, `calcTotalNumberOfStates()`

23

Sgd.h / Sgd.c`src/optimizer/include/Sgd.h + src/optimizer/Sgd.c`

Implements Stochastic Gradient Descent. `sgd_t` holds `learningRate`, `momentumFactor`, and `weightDecay`. `sgdStep()` applies $w \leftarrow lr * grad + weightDecay * w$. `sgdStepM()` is the momentum variant: $v = momentum * v + grad$; $w \leftarrow lr * v$. `sgdZeroGrad()` zeros all gradient tensors in the parameter array after each step. Both float32 and int8 quantization paths are handled.

Reads from: `Optimizer.h`, `Tensor.h`.

Key types/functions: `sgd_t`, `sgdStep()`, `sgdStepM()`, `sgdZeroGrad()`

24

SgdApi.h / SgdApi.c + OptimizerApi.h / OptimizerApi.c`src/userApi/optimizer/include/SgdApi.h + src/userApi/optimizer/SgdApi.c`

`sgdMCreateOptim(lr, momentum, weightDecay, model, sizeModel, qType)` is the single call used in experiments to create a fully initialised optimizer. It traverses the model layer array, collects all `parameter_t` pointers, allocates velocity buffers via `reserveMemory()`, and wires them into an `optimizer_t`. `freeOptimSgdM()` tears it down. `OptimizerApi.c` provides the generic `optimizerInit()` wrapper.

Reads from: `SgdApi.h`, `Optimizer.h`, `Layer.h`, `StorageApi.h`.

Key types/functions: `sgdMCreateOptim()`, `freeOptimSgdM()`

Phase 9

Data I/O

*Dataset structs · .npy loader ·
CSV logger · DataLoader*

25

Dataset.h`src/data_loader/include/Dataset.h (header only)`

Pure struct definitions — no functions. Defines: `sample_t` (item tensor + label tensor), `batch_t` (array of `sample_t` pointers + size), `tensorArray_t` (array of `tensor_t` pointers + size), and `dataset_t` (two `tensorArrays`: items and labels). These are the data containers passed throughout the training loop.

Reads from: `Tensor.h`.

Key types/functions: `sample_t`, `batch_t`, `tensorArray_t`, `dataset_t`

26

NPYLoader.h / NPYLoader.c`src/data_loader/include/NPYLoader.h + src/data_loader/NPYLoader.c`

Low-level .npy file parser. Implements: `openNPYFile()`, `checkMagic()` (validates the `\x93NUMPY` magic bytes), `readHeaderSize()`, `readHeader()` (reads the ASCII header string), `getDTypeFromHeader()` (parses `<f4` or `<i4` dtype strings), `getNumberOfDimsFromHeader()`, and `getShapeFromHeader()` (fills a `shape_t` from the header tuple). These are the building blocks used by the `NPYLoaderApi`.

Reads from: `Dataset.h`, `Tensor.h`.

Key types/functions: `openNPYFile()`, `checkMagic()`, `getDTypeFromHeader()`, `getShapeFromHeader()`

27

NPYLoaderApi.h / NPYLoaderApi.c`src/userApi/data_loader/include/NPYLoaderApi.h + NPYLoaderApi.c`

High-level .npy loading API. `npLoad(path)` opens a .npy file, parses its header, reads all data into a `tensorArray_t` (one tensor per sample), and returns it.

`npGetSample(dataset, index)` returns a `sample_t` pointing into the loaded arrays.

Used in `MnistExperiment.c` as: `trainItems = npLoad(MNIST_TRAIN_X)`.

Reads from: `NPYLoader.h`, `TensorApi.h`, `StorageApi.h`.

Key types/functions: `npLoad()`, `npGetSample()`

28

DataLoader.h / DataLoader.c`src/data_loader/include/DataLoader.h + src/data_loader/DataLoader.c`

Core `dataLoader_t` struct and `initDataLoader()`. The struct holds function pointers `getSample`, `getDatasetSize`, `getBatch`, `transform`, `targetTransform` plus configuration fields `batchSize`, `shuffle`, `shuffleSeed`, `indices[]`, and `dropLast`. `initDataLoader()` wires all fields. `getBatch` is a user-supplied function (or the default from `DataLoaderApi`) that assembles `batch_t` structs from individual samples.

Reads from: `Dataset.h`, `Tensor.h`.

Key types/functions: `dataLoader_t`, `initDataLoader()`, `getSampleFn_t`, `getBatchFn_t`

29

DataLoaderApi.h / DataLoaderApi.c`src/userApi/data_loader/include/DataLoaderApi.h + DataLoaderApi.c`

`dataLoaderInit()` is the user-facing constructor — it calls `reserveMemory()` for the `dataLoader_t` and the indices array, optionally shuffles using RNG, and calls `initDataLoader()`. Also provides `freeSample()`, `freeBatch()`, and `freeDataLoader()` for cleanup. This is the only call you need to create a data loader.

Reads from: `DataLoader.h`, `StorageApi.h`, `RNG.h`.

Key types/functions: `dataLoaderInit()`, `freeSample()`, `freeBatch()`, `freeDataLoader()`

30

CSVHelper.h / CSVHelper.c`src/csv/include/CSVHelper.h + src/csv/CSVHelper.c`

A minimal CSV writer/reader. `csvData_t` holds a `char**` rows array, a row count, and a per-row entry count. `setCSVData()` fills the struct. `csvWriteRowsByBufferSize()` appends or overwrites rows in a file (mode 'a' or 'w'). `csvReadRowsByBufferSize()` reads rows back in. `csvParseBufferAsFloat()` converts string tokens to floats. Used in `MnistExperiment.c` to log epoch metrics.

Reads from: *Nothing (uses stdio).*

Key types/functions: `csvData_t`, `setCSVData()`, `csvWriteRowsByBufferSize()`, `csvParseBufferAsFloat()`

Phase 10

Serialization

Binary save and load of model weights

31

Serialize.h / Serialize.c`src/serial/include/Serialize.h + src/serial/Serialize.c`

Writes model state to a binary FILE*. `serializeTensor()` writes the tensor's shape (ndims, each dim), quantization config, and raw data bytes using the DTypes byte-write helpers. `serializeParameter()` calls `serializeTensor()` twice (param + grad). `serializeModel()` iterates the layer array, identifies trainable layers, and serializes their parameters. Enables checkpointing and UART weight transfer to MCU.

Reads from: *Serialize.h, Tensor.h, DTypes.h.*

Key types/functions: `serializeTensor()`, `serializeParameter()`, `serializeModel()`

32

Deserialize.h / Deserialize.c`src/serial/include/Deserialize.h + src/serial/Deserialize.c`

Mirror of `Serialize`. `deserializeTensor()` reads shape and data from a FILE* into an already-allocated `tensor_t` (the caller must have pre-allocated the struct). `deserializeParameter()` and `deserializeModel()` follow the same pattern. Used to resume training on the MCU after a power cycle, or to load a host-pretrained model onto the device.

Reads from: *Deserialize.h, Tensor.h, DTypes.h.*

Key types/functions: `deserializeTensor()`, `deserializeParameter()`, `deserializeModel()`

Phase 11

Inference

Forward-only pass and batched evaluation

33

InferenceApi.h / InferenceApi.c`src/userApi/include/InferenceApi.h + src/userApi/InferenceApi.c`

Three inference modes. `inference(model, nLayers, input)` runs a single forward pass and returns the final output tensor. `inferenceBatched()` runs inference over a whole `batch_t` and returns an array of output tensors. `inferenceWithLoss()` runs forward + loss forward and returns an `inferenceStats_t` holding the output tensor and the scalar loss. This is the function passed to the training loop for evaluation.

Reads from: `Layer.h`, `Tensor.h`, `LossFunction.h`, `DataLoader.h`.

Key types/functions: `inference()`, `inferenceBatched()`, `inferenceWithLoss()`, `inferenceStats_t`

Phase 12

Training Loop

*Backprop · epoch/batch
orchestration · metrics*

34

CalculateGradsSequential.h / CalculateGradsSequential.c`src/userApi/training_loop/calculate_grads/include/CalculateGradsSequential.h + .c`

Implements `calculateGradsSequential()` — the backpropagation engine. Given the model, a single input tensor, and a label: (1) runs the full forward pass storing intermediate activations, (2) computes the loss, (3) walks the layer array in reverse calling each layer's backward function with the upstream gradient, (4) returns a `trainingStats_t` with the output and loss. This matches the signature of `calculateGradsFn_t`, so it can be swapped for a sparse or quantized variant later.

Reads from: `TrainingLoopApi.h`, `Layer.h`, `LossFunction.h`, `InferenceApi.h`.

Key types/functions: `calculateGradsSequential()`

35

TrainingBatchDefault.h / TrainingBatchDefault.c`src/userApi/training_loop/training_batch/include/TrainingBatchDefault.h + .c`

`trainingBatchDefault()` processes one `batch_t`: iterates all samples in the batch, calls `calculateGradsFn` (e.g. `calculateGradsSequential`) for each, accumulates gradients (sum over the batch), calls `optimizer step()`, then zeros gradients. Returns the mean batch loss as a float.

Reads from: `TrainingLoopApi.h`, `Optimizer.h`.

Key types/functions: `trainingBatchDefault()`

36

TrainingEpochDefault.h / TrainingEpochDefault.c`src/userApi/training_loop/training_epoch/include/TrainingEpochDefault.h + .c`

`trainingEpochDefault()` runs one full epoch: calls the `DataLoader`'s `getBatch()` in a loop, passes each batch to `trainingBatchDefault()`, accumulates and returns the mean epoch training loss. Handles `dropLast` and optional shuffle (via the `dataloader`'s `index` array).

Reads from: `TrainingLoopApi.h`, `DataLoader.h`.

Key types/functions: `trainingEpochDefault()`

37

TrainingLoopApi.h / TrainingLoopApi.c

[src/userApi/training_loop/include/TrainingLoopApi.h](#) + [TrainingLoopApi.c](#)

The top-level training orchestrator. Defines all result structs: `trainingStats_t`, `epochStats_t` (loss, accuracy, precision, recall, f1), `classificationReport_t` (epoch stats + confusion matrix), `trainingRunResult_t` (final loss + eval stats). Key functions: `evaluationEpochWithMetrics()` — full eval pass with macro P/R/F1; `evaluationEpochWithReport()` — eval pass with confusion matrix; `trainingRun()` — the main entry point that runs `numberOfEpochs` training epochs, evaluates after each, and calls the `epochCallbackFn_t` hook.

Reads from: *Tensor.h, Optimizer.h, LossFunction.h, DataLoader.h, InferenceApi.h.*

Key types/functions: *trainingRun(), epochStats_t, classificationReport_t, trainingRunResult_t, epochCallbackFn_t*

Phase 13

Experiment

*End-to-end MNIST validation —
read last*

38

MnistExperiment.c

[experiments/MnistExperiment.c](#)

The end-to-end validation experiment. Read this file last — it ties together every module you have read so far. Structure: (1) `initDataSets()` loads MNIST train/test splits via `numpyLoad()`; (2) `flattenItemDims()` reshapes 28x28 images to 784-vectors; (3) `buildModel()` constructs the 4-layer model (Linear 784->20, ReLU, Linear 20->10, Softmax) using `TensorApi` and the layer constructors; (4) two `DataLoaders` are created; (5) `sgdMCreateOptim()` creates the optimizer; (6) `trainingRun()` is called with `calculateGradsSequential` and `inferenceWithLoss`; (7) the `epochCallback` writes every epoch's metrics (loss, accuracy, P, R, F1) to a CSV log.

Reads from: *All modules above.*

Key types/functions: *All of the above — this file is the integration test for the whole codebase.*

Quick Reference — All Files in Reading Order

#	File(s)	Layer	Key export
1	common/include/Common.h	Foundation	PRINT_DEBUG/INFO/ERROR
2	arithmetic/Rounding.h/.c	Foundation	roundingMode_t, roundByMode()
3	tensor/DTypes.h/.c	Foundation	readBytesAsInt32/Float(), write*()
4	tensor/DataStorage.h/.c	Memory	dataStorage_t, addDataToStorage()
5	userApi/StorageApi.h/.c	Memory	reserveMemory(), freeReservedMemory()
6	tensor/Quantization.h/.c	Tensor	quantization_t, qtype_t, init*Quantization()
7	tensor/Tensor.h/.c	Tensor	tensor_t, parameter_t, shape_t, transposeTensor()
8	tensor/TensorConversion.h/.c	Tensor	convertTensor(), conversionMatrix[][]
9	userApi/tensor/QuantizationApi.h/.c	Tensor	quantizationInitFloat/SymInt32/Asym()
10	userApi/tensor/TensorApi.h/.c	Tensor	tensorInitWithDistribution(), parameterInit(), freeTensor()
11	arithmetic/Arithmetic.h/.c	Arithmetic	int32/floatPointWiseArithmetic(), calcTensorIndexByIndices()
12	arithmetic/Matmul.h/.c	Arithmetic	matmulFloat32Tensors(), matmulSymInt32Tensors()
13	arithmetic/Add,Sub,Mul,Div,Log,Sum,Square,...h/.c	Arithmetic	Element-wise ops, argmax(), xavierUniform()
14	rng/RNG.h/.c	RNG	rngSetSeed(), rngNextFloat(), rngShuffleIndices()
15	layer/Layer.h/.c	Layer	layer_t, layerType_t, layerFunctions[], initLayer()
16	layer/Linear.h/.c	Layer	linearConfig_t, linearForward/Backward*()
17	layer/Relu.h/.c	Layer	reluForward/BackwardFloat/Asym()
18	layer/Softmax.h/.c	Layer	softmaxForward/Backward()
19	layer/QuantizationLayer.h/.c	Layer	quantizationForward/Backward() — FQT hook
20	layer/Conv1d.h/.c (stub)	Layer	(planned — not yet implemented)
21	userApi/layer/LinearApi,ReluApi,SoftmaxApi.h/.c	Layer API	linearLayerInit(), reluLayerInit(), softmaxLayerInit()
22	loss_functions/LossFunction.h/.c	Loss	lossType_t, lossFunctions[]

2 3	loss_functions/CrossEntropy.h/.c	Loss	crossEntropyForwardFloat(), crossEntropySoftmaxBackward()
2 4	loss_functions/MSE.h/.c	Loss	mseLossForward(), mseLossBackwardFloat()
2 5	optimizer/Optimizer.h/.c	Optimizer	optimizer_t, optimizerFunctions[], calcTotalNumberOfStates()
2 6	optimizer/Sgd.h/.c	Optimizer	sgd_t, sgdStep(), sgdStepM(), sgdZeroGrad()
2 7	userApi/optimizer/SgdApi.h/.c	Optimizer API	sgdMCreateOptim(), freeOptimSgdM()
2 8	data_loader/Dataset.h	Data I/O	sample_t, batch_t, tensorArray_t, dataset_t
2 9	data_loader/NPYLoader.h/.c	Data I/O	openNPYFile(), getDTypeFromHeader(), getShapeFromHeader()
3 0	userApi/data_loader/NPYLoaderApi.h/.c	Data I/O	npLoad(), npGetSample()
3 1	data_loader/DataLoader.h/.c	Data I/O	dataLoader_t, initDataLoader()
3 2	userApi/data_loader/DataLoaderApi.h/.c	Data I/O	dataLoaderInit(), freeBatch()
3 3	csv/CSVHelper.h/.c	Data I/O	csvData_t, csvWriteRowsByBufferSize()
3 4	serial/Serialize.h/.c	Serialization	serializeModel(), serializeTensor()
3 5	serial/Deserialize.h/.c	Serialization	deserializeModel(), deserializeTensor()
3 6	userApi/InferenceApi.h/.c	Inference	inference(), inferenceWithLoss(), inferenceBatched()
3 7	training_loop/calculate_grads/CalculateGradsSequential.h/.c	Training	calculateGradsSequential()
3 8	training_loop/training_batch/TrainingBatchDefault.h/.c	Training	trainingBatchDefault()
3 9	training_loop/training_epoch/TrainingEpochDefault.h/.c	Training	trainingEpochDefault()
4 0	training_loop/TrainingLoopApi.h/.c	Training	trainingRun(), epochStats_t, classificationReport_t
4 1	experiments/MnistExperiment.c	Experiment	main() — end-to-end integration

All source files are under `src/` relative to the `OnDeviceTraining/` project root. The `experiments/` directory is at the same level as `src/`. Header (`.h`) files define the public contract; read them before the `.c` implementation. Internal headers (`*Internal.h`) are implementation details — you can read them after understanding the public API.